

Java Backend Architecture

Interview Series

Chapter 14.7

Database at Scale - Sharding & CQRS

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 14.7 - Database at Scale: Sharding & CQRS

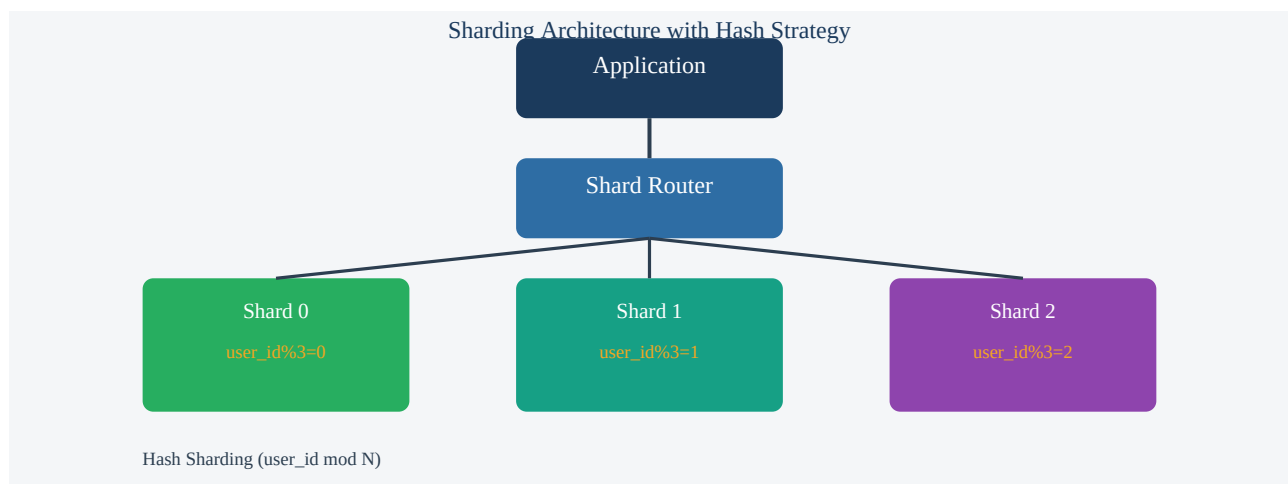
Overview

Scaling databases beyond a single server requires horizontal partitioning (sharding), read scaling via replicas, and architectural patterns like CQRS that separate read and write paths. Distributed transactions introduce coordination challenges addressed by Saga, 2PC, and TCC patterns.

Read Replicas

- Route SELECT queries to read replicas; writes go to the primary
- Replication lag: replicas are eventually consistent - reads may return stale data
- Acceptable for: user feeds, product listings, analytics queries
- Not acceptable for: read-after-write consistency (e.g., read own write)
- Session consistency workaround: route reads for a user to primary for a short window after a write

Sharding Architecture



Sharding Strategies

Strategy	Mechanism	Pros	Cons
Hash Sharding	shard = hash(key) % N	Even distribution, predictable	Resharding requires remapping all keys
Range Sharding	shard = key range (A-M, N-Z)	Range queries efficient	Hotspot risk (sequential keys)
Directory Sharding	Lookup table: key -> shard_id	Flexible, custom mapping	Lookup table is single point of failure
Geo Sharding	Shard by region/country	Data locality, compliance	Cross-region queries expensive
Consistent Hashing	Keys on a ring, N virtual nodes	Minimal resharding on scale	Complexity, uneven load without vnodes

Hotspot Avoidance

- Avoid sequential shard keys (auto-increment IDs) with range sharding - all writes go to last shard
- Use hash sharding or UUIDs to distribute writes evenly
- For celebrities/hot keys: add random suffix to shard key, or dedicate extra shards

- Monitor shard size and query distribution; rebalance via consistent hashing

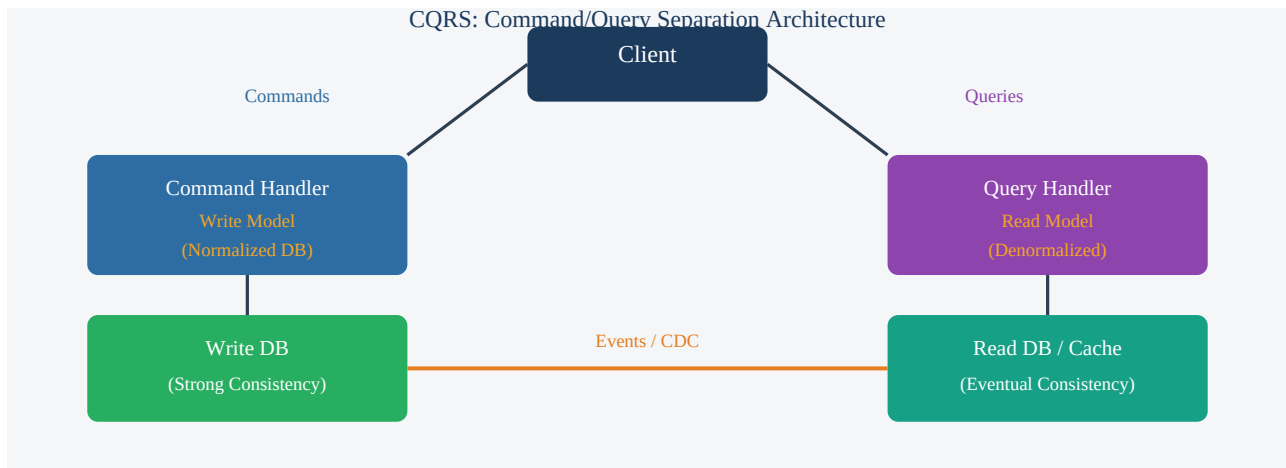
Cross-Shard Queries

- Scatter-gather: query all shards in parallel, merge results in application layer
- Avoid joins across shards - denormalize data or use a reference service
- Aggregations (COUNT, SUM): gather partial results from each shard, aggregate in app
- Global secondary index: maintain a separate index shard mapping secondary key to primary shard

Sharding in Spring Data / JPA

```
// Custom shard routing with AbstractRoutingDataSource public class ShardRoutingDataSource
extends AbstractRoutingDataSource { @Override protected Object determineCurrentLookupKey() {
Long userId = ShardContext.getCurrentUserId(); return "shard_" + (userId % 3); // 3 shards } }
// ShardContext (ThreadLocal) public class ShardContext { private static final
ThreadLocal<Long> context = new ThreadLocal<>(); public static void setUserId(Long id) {
context.set(id); } public static Long getCurrentUserId() { return context.get(); } public
static void clear() { context.remove(); } } // Usage in service ShardContext.setUserId(userId);
try { return orderRepository.findById(userId); } finally { ShardContext.clear(); }
```

CQRS - Command Query Responsibility Segregation



CQRS Concepts

Command	Write operation that changes state. Uses normalized write model with strong consistency. Returns void or ID.
Query	Read operation that returns data. Uses denormalized read model for fast retrieval. Eventually consistent.
Read Model	Pre-computed, denormalized view optimized for query patterns. Can be a separate DB, Elasticsearch, Redis.
Event Sync	Commands publish domain events; a projection updates the read model asynchronously.
CDC	Change Data Capture (Debezium) captures write DB changes and feeds the read model without explicit event publishing.

CQRS Implementation with Spring

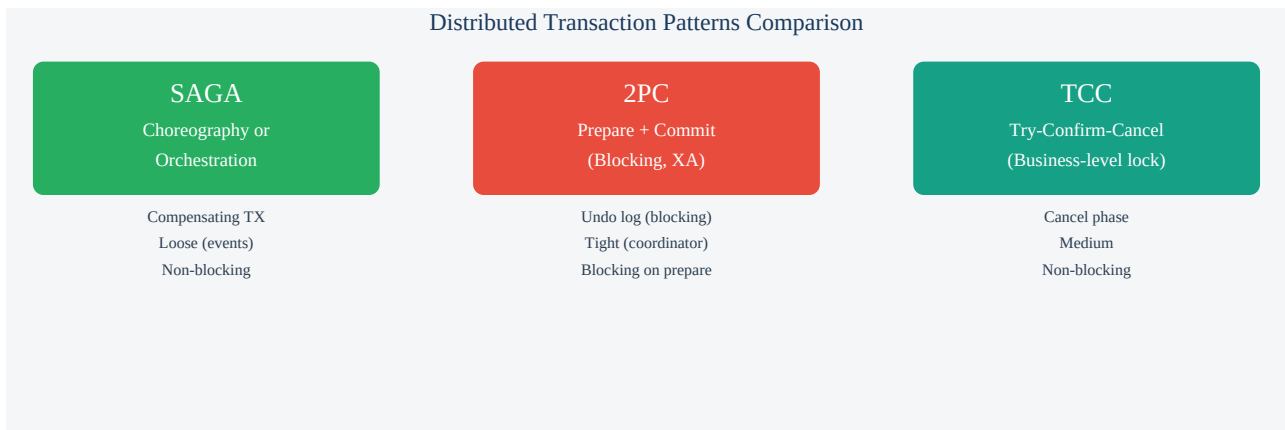
```
// Command side - write to normalized DB @Service public class OrderCommandService {  
    @Transactional public UUID createOrder(CreateOrderCommand cmd) { Order order = new  
        Order(cmd.getUserId(), cmd.getItems()); orderRepository.save(order); // Publish domain event  
        eventPublisher.publish(new OrderCreatedEvent(order.getId(), order)); return order.getId(); } }  
// Event listener - update read model @Component public class OrderProjection { @EventListener  
    @Async public void on(OrderCreatedEvent event) { OrderReadModel readModel =  
        OrderReadModel.from(event.getOrder()); orderReadRepository.save(readModel); // denormalized,  
        in Elasticsearch or read DB } } // Query side - read from denormalized model @Service public  
class OrderQueryService { public List<OrderSummaryDto> getUserOrders(UUID userId) { return  
    orderReadRepository.findById(userId); // fast read } }
```

Event Sourcing with CQRS

- Store the full sequence of domain events as the source of truth, not current state
- Replay events to rebuild current state or build new read projections
- Audit trail: every state change is an immutable event record
- Snapshots: periodically snapshot aggregate state to avoid replaying all events
- Tools: Axon Framework (Java), EventStoreDB, Kafka as event log

Distributed Transactions: Saga, 2PC, TCC

Distributed Transaction Patterns Comparison



Pattern	Mechanism	Rollback	Blocking	Use Case
Saga	Sequence of local TXs, each publishes compensating transactions	Compensating transactions	Non-blocking	Microservice workflows
2PC	Prepare phase + Commit phase via coordinator	Undo log (auto rollback)	Blocking on prepare	Single-DB distributed TX (XA)
TCC	Try (reserve) + Confirm or Cancel	Cancel phase	Non-blocking	Payment, booking systems

Saga: Choreography vs Orchestration

Choreography	Each service reacts to events and publishes its own events. Decentralised, loose coupling. Hard to visualise overall flow.
Orchestration	A central Saga orchestrator (e.g., Axon Saga, Temporal workflow) directs each step. Easier to monitor, centralised logic.
Compensating TX	Each step has a compensation action (e.g., Cancel Payment reverses Charge Payment). Must be idempotent.
Failure handling	If step N fails, execute compensations for steps N-1 down to 1 in reverse order.

Saga Orchestration Example

```
// Axon Saga Orchestrator (simplified)
@Saga public class OrderSaga {
    @StartSaga
    @SagaEventHandler(associationProperty = "orderId") public void on(OrderCreatedEvent event) {
        commandGateway.send(new ReserveInventoryCommand(event.getOrderId(), event.getItems()));
    }
    @SagaEventHandler(associationProperty = "orderId") public void on(InventoryReservedEvent event) {
        commandGateway.send(new ChargePaymentCommand(event.getOrderId(), event.getAmount()));
    }
    @SagaEventHandler(associationProperty = "orderId") public void on(PaymentFailedEvent event) {
        // compensate
        commandGateway.send(new ReleaseInventoryCommand(event.getOrderId()));
        commandGateway.send(new CancelOrderCommand(event.getOrderId()));
    }
    @EndSaga
    @SagaEventHandler(associationProperty = "orderId") public void on(OrderCompletedEvent event) {
    }
}
```

Interview Q&A - Database at Scale: Sharding & CQRS

Q1. What is database sharding and why is it needed?

Sharding horizontally partitions data across multiple database instances (shards) to handle data volumes or write throughput that exceed a single server's capacity. Each shard holds a subset of rows determined by a shard key. Unlike vertical scaling (bigger hardware), sharding scales linearly by adding shards.

Q2. What makes a good shard key?

A good shard key: (1) distributes data and queries evenly to avoid hotspots; (2) matches common query patterns (queries should hit one shard, not all); (3) does not change after insertion; (4) has high cardinality. Common choices: `user_id` (hash), `order_id`, `tenant_id`. Avoid: `timestamp` (range hotspot), `status` (low cardinality).

Q3. Compare hash sharding and range sharding.

Hash sharding: $\text{shard} = \text{hash}(\text{key}) \% N$. Even data distribution, no hotspots, but range queries scatter across all shards and resharding requires remapping. Range sharding: $\text{shard} = \text{key range}$ (e.g., `user_id` 0-999 on shard 0, 1000-1999 on shard 1). Efficient range queries but risk of hotspots on monotonically increasing keys (all writes to last shard).

Q4. What is a cross-shard query and how do you handle it?

A cross-shard query requires data from multiple shards (e.g., `COUNT(*)` across all users). Handled via scatter-gather: query all shards in parallel, gather partial results in application layer, merge/aggregate. Avoid cross-shard joins by denormalizing data. For reporting, use a separate analytics DB (Redshift, BigQuery) fed by CDC.

Q5. What is directory-based sharding?

A lookup service (directory) maps each key to its shard: `shard_id = directory.lookup(key)`. Very flexible - supports arbitrary distribution, easy resharding (update directory). Downside: the directory is a critical component and potential bottleneck/single-point-of-failure; needs caching and HA setup.

Q6. What is consistent hashing and how does it help with resharding?

Keys and nodes are placed on a hash ring ($0..2^{32}$). A key maps to the first node clockwise on the ring. Adding a node: only keys between the new node and its predecessor move. Removing a node: its keys move to the next node. Only $1/N$ of keys need remapping (vs all keys in naive hash sharding). Virtual nodes ensure even distribution.

Q7. How does replication lag affect read replicas?

Replication lag is the delay between a write on the primary and its visibility on the replica. Under high write load, lag can grow to seconds or more. Consequences: reads may return stale data (eventual consistency). Mitigation: read-after-write consistency by routing the user's own reads to primary for a brief window; monitor lag via `pg_stat_replication` or `Seconds_Behind_Master` in MySQL.

Q8. What is CQRS?

Command Query Responsibility Segregation separates the write model (Commands - create/update/delete, strong consistency, normalized DB) from the read model (Queries - read-only, eventual consistency, denormalized for fast retrieval). They are separate code paths and can use different storage technologies. Synchronization happens via domain events or CDC.

Q9. What are the benefits of CQRS?

Independent scaling: read models can be replicated widely without affecting writes. Optimized models: read DB can be denormalized, materialized views, or Elasticsearch for full-text search. Performance: queries never contend with writes. Flexibility: multiple read models for different query patterns. Drawback: eventual consistency between write and read models, increased complexity.

Q10. How do you synchronize the read model in CQRS?

Two main approaches: (1) Domain events - command side publishes events (OrderCreatedEvent) to a message bus (Kafka); read model projection consumes events and updates read DB. (2) CDC (Change Data Capture) - Debezium captures write DB transaction log changes and streams them to a Kafka topic; projection updates read model without modifying the service code.

Q11. What is event sourcing and how does it relate to CQRS?

Event sourcing stores the sequence of domain events as the source of truth rather than current state. Current state is derived by replaying events. CQRS pairs naturally: the event log is the write model (append-only, immutable); projections build multiple read models from the event stream. Benefits: full audit trail, time travel (replay to any point), new projections anytime.

Q12. What is a Saga and when would you use it?

A Saga is a sequence of local transactions across multiple services, each publishing events that trigger the next step. If a step fails, compensating transactions undo previous steps. Use when: a business operation spans multiple microservices with separate databases (e.g., order creation touching inventory, payment, shipping), making 2PC impractical.

Q13. Compare Saga choreography and Saga orchestration.

Choreography: each service reacts to events and emits its own events. Decentralised, loosely coupled, but hard to visualise overall flow and debug failures. Orchestration: a central Saga orchestrator (Axon Saga, Temporal) sends commands to each step and handles failures centrally. Easier to monitor, test, and reason about. Adds a central component that can be a bottleneck.

Q14. What is a compensating transaction?

A compensating transaction is a business-level operation that reverses the effect of a previously completed local transaction. For example, if "Charge Payment" succeeded but "Reserve Inventory" failed, the compensation is "Refund Payment". Compensating transactions must be idempotent (safe to retry) and must handle partial completion (e.g., partial shipment already sent).

Q15. What is Two-Phase Commit (2PC)?

2PC has two phases: (1) Prepare - coordinator asks all participants to prepare and lock resources, each votes yes/no. (2) Commit/Abort - if all vote yes, coordinator sends commit; any no vote triggers abort. Ensures atomicity across distributed DBs. Drawbacks: blocking (participants hold locks during prepare), coordinator is single point of failure, poor performance under high concurrency.

Q16. What is TCC (Try-Confirm-Cancel)?

TCC is a two-phase commit variant at the business logic level: (1) Try - reserve/lock resources without committing (e.g., put seat in HELD status). (2) Confirm - commit the reservation (seat becomes BOOKED). (3) Cancel - release the reservation if any step fails (seat becomes AVAILABLE). Unlike 2PC, locking is at the business object level, not DB row locks - non-blocking.

Q17. Why is 2PC problematic in microservices?

2PC requires all participants to be available simultaneously (tight coupling), holds database locks during the prepare phase (blocks concurrent reads/writes), and the coordinator is a single point of failure (if it crashes between prepare and commit, participants are stuck holding locks). In microservices with network partitions (CAP theorem), 2PC may block indefinitely.

Q18. What is the hotspot problem in sharding?

A hotspot occurs when one shard receives disproportionately more traffic than others. Causes: (1) sequential shard key (all new inserts go to the last range shard); (2) a single very popular user/entity (celebrity problem); (3) low-cardinality shard key (e.g., sharding by gender - only 2 values). Solutions: use hash sharding, add random prefix, dedicated shard for hot keys, or application-level caching.

Q19. How do you handle resharding?

Steps: (1) double-write to old and new sharding scheme during migration; (2) backfill data to new shards; (3) verify consistency; (4) switch reads to new scheme; (5) remove old shards. Consistent hashing minimizes data movement (only $\sim 1/N$ keys move). Tools: Vitess for MySQL, Citus for PostgreSQL handle resharding transparently.

Q20. What is a read model in CQRS and how is it structured?

The read model is a denormalized, query-optimized view of data. It pre-joins related tables, pre-computes aggregations, and stores exactly the fields needed by each query. For example: UserOrderSummary table with user_name, total_orders, last_order_date (pre-computed from joins). Can be stored in: relational DB (materialized views), Elasticsearch, Redis, or DynamoDB.

Q21. What is CDC (Change Data Capture) and how does it work?

CDC captures row-level changes (INSERT/UPDATE/DELETE) from the database transaction log (WAL in PostgreSQL, binlog in MySQL) and streams them as events. Debezium is the standard tool: it tails the log and publishes changes to Kafka topics per table. Consumers (projections) update read models or search indices without modifying the source service code.

Q22. How do you ensure idempotency in event-driven CQRS projections?

Each event should carry a unique event ID. The projection records processed event IDs (in a processed_events table or Redis set). Before processing, check if the event ID was already handled - skip if so. This handles message redelivery (Kafka at-least-once). Alternatively, design projections to be naturally idempotent (upsert operations with last-wins semantics).

Q23. What is a materialized view and how does it relate to CQRS read models?

A materialized view stores the result of a query as a physical table, updated on schedule or on trigger. In PostgreSQL: CREATE MATERIALIZED VIEW order_summary AS SELECT ... REFRESH MATERIALIZED VIEW CONCURRENTLY. The CQRS read model is a more flexible version - updated in real-time via events or CDC rather than batch refresh, and can span multiple tables or even services.

Q24. What database technologies are commonly used for CQRS read models?

Depends on query pattern: Elasticsearch (full-text search, faceted filtering, complex aggregations), Redis (low-latency key-value lookups, sorted sets for rankings), PostgreSQL with denormalized tables or materialized views (complex SQL queries), Cassandra (time-series, append-heavy reads), DynamoDB (simple key lookups at scale). Multiple read models can coexist for different use cases.

Q25. What are Axon Framework and Temporal for distributed transactions?

Axon Framework provides CQRS+Event Sourcing infrastructure for Java/Spring: command bus, event store, Saga support, projections. Events stored in AxonServer or DB. Temporal is a workflow orchestration platform: workflows are code that survives failures (durable execution), with built-in retry, compensation, and Saga patterns. Both abstract distributed transaction complexity from business logic.

Q26. How does Vitess help with MySQL sharding?

Vitess is a database clustering system built on MySQL, used by YouTube/Slack. It provides: transparent sharding (application does not know about shards), connection pooling (VTTablet), query routing (VTGate), online schema changes, resharding without downtime, and cross-shard query support. The application connects to Vitess as if it were a single MySQL instance.

Q27. What is the N+1 query problem and how does sharding affect it?

N+1: fetching 1 parent + N child queries in a loop. In a sharded environment, this becomes worse: each child query may hit a different shard, multiplying network round-trips. Solutions: use batch queries (IN clause with all IDs), denormalize data to avoid child queries, use DataLoader pattern (batch + deduplicate), or design shard key to co-locate parent and children on the same shard.

Q28. How do you handle global unique IDs across shards?

Options: (1) UUID v4 - globally unique but not sequential (index fragmentation). (2) Snowflake ID - 64-bit: timestamp (41 bits) + datacenter/machine ID (10 bits) + sequence (12 bits). Monotonically increasing, distributed, no coordination needed. (3) ULID - URL-safe, lexicographically sortable UUID. (4) Database sequence per shard with shard prefix (e.g., shard_1:1001).

Q29. What is the difference between vertical and horizontal scaling for databases?

Vertical scaling (scale-up): add more CPU, RAM, faster disk to a single server. Simple but has physical limits and single point of failure. Horizontal scaling (scale-out): add more servers. Sharding enables horizontal write scaling; read replicas enable horizontal read scaling. More complex (distributed coordination, cross-shard queries) but theoretically unlimited scale.

Q30. How do you handle distributed transactions without 2PC in microservices?

Use eventual consistency with Saga pattern: each service commits its local transaction and publishes a domain event; the Saga coordinates the workflow and issues compensating transactions on failure. Accept that the system is temporarily inconsistent between steps. Design for idempotency and compensatability. Use outbox pattern to guarantee event delivery atomically with the local DB write.

Q31. What is the Outbox Pattern?

The Outbox Pattern atomically writes to the database and publishes an event by: writing the business record AND an outbox event record in the same local transaction. A separate Outbox Processor (or Debezium CDC) reads the outbox table and publishes events to Kafka. This avoids the dual-write problem (DB commit + Kafka publish can fail independently), ensuring exactly-once event publishing.

Q32. What is a projection in event sourcing?

A projection reads the event stream and builds a materialized view (read model). Each event is processed in order to update the projection state. Multiple projections can be built from the same events for different query needs. Projections can be rebuilt from scratch by replaying all events. In Axon: @EventHandler methods in a @Projection class update a JPA read model.

Q33. How do you handle schema evolution in event sourcing?

Events are immutable once stored. For schema changes: (1) Upcasting - transform old events to new format at read time (Axon Upcasters). (2) Versioning - store event version, handle each version in projections. (3) Additive changes only - add new optional fields, never remove/rename. (4) Snapshot with new schema - take a snapshot after schema change to avoid replaying old-format events.

Q34. What is the difference between CQRS and simple read replicas?

Read replicas are a database-level concern: the same data model is replicated for read scaling. CQRS is an architectural pattern: the write model (normalized) and read model (denormalized, optimized) are fundamentally different data structures in potentially different storage technologies. CQRS enables query-specific optimization that read replicas cannot provide.

Q35. How does the Saga handle the case where a compensating transaction also fails?

This is a challenging edge case. Options: (1) Retry with backoff - most compensations are idempotent and eventually succeed. (2) Dead letter queue - failed compensations go to a DLQ for manual intervention. (3) Alert on compensation failure - human operators must resolve. (4) Design compensations to be very simple and reliable (avoid external calls). (5) Use TCC where cancel is a DB status update and highly reliable.

Q36. What is sharding vs partitioning?

Partitioning splits data across multiple tables or files within the same database instance (e.g., PostgreSQL table partitioning by date). Sharding splits data across multiple database instances/servers. Partitioning improves query performance via partition pruning without distributed coordination overhead. Sharding provides true horizontal scaling across machines but adds distributed complexity.

Q37. How do you perform online schema changes on a sharded database?

Apply changes shard-by-shard to minimize impact. For MySQL: use pt-online-schema-change (Percona Toolkit) or gh-ost which create a shadow table, mirror writes, backfill data, then swap atomically. For PostgreSQL: use pg_repack or declarative partitioning. With Vitess: use VTAdmin managed schema changes that handle propagation across shards. Plan for backward compatibility during rollout.

Q38. What is the difference between synchronous and asynchronous CQRS?

Synchronous CQRS: after writing, immediately update the read model in the same transaction (or via a synchronous call). Ensures immediate read-your-writes consistency but couples write and read performance. Asynchronous CQRS: write model publishes events; read model updates asynchronously. Higher throughput, lower coupling, but introduces eventual consistency lag (typically milliseconds to seconds).

Q39. How do you choose between Saga and TCC?

Use Saga when: steps are long-running, business processes involve multiple services, eventual consistency is acceptable, and compensating actions are well-defined. Use TCC when: precise resource reservation is required (hotel rooms, flight seats), the reservation window is short and bounded, and you need stronger consistency than Saga's eventual model. TCC is more complex to implement correctly (idempotent try/cancel phases).

Q40. What is a CRDT and is it relevant to distributed databases?

Conflict-free Replicated Data Types (CRDTs) are data structures designed to merge concurrent updates without conflicts. Relevant in: eventually consistent databases (Riak, DynamoDB with DAX), collaborative editing, shopping carts. Examples: G-Counter (grow-only), OR-Set (add/remove set). In Cassandra: counters and LWW (last-write-wins) provide similar semantics. CRDTs guarantee convergence without coordination.

Q41. How does PostgreSQL table partitioning work?

PostgreSQL supports declarative partitioning: RANGE (by date, ID range), LIST (by category, region), and HASH (by mod). CREATE TABLE orders PARTITION BY RANGE (created_at); with child tables for each range. Queries against the parent table use partition pruning to scan only relevant partitions. Partition maintenance (add new month partition, drop old) is online and non-blocking.

Q42. What is the role of Kafka in a CQRS+Event Sourcing architecture?

Kafka acts as the durable, ordered event log between the command side and read model projections. Properties that make it suitable: (1) log compaction preserves latest event per key; (2) consumer groups allow multiple independent projections to consume the same events; (3) replay - consumers can reset offset to replay all events and rebuild projections; (4) ordering per partition ensures event sequence integrity.

Q43. How do you handle eventual consistency in the UI for CQRS?

Strategies: (1) Optimistic UI - update the UI immediately with the expected state after a command, then confirm from the read model when it updates. (2) Poll with timeout - after issuing a command, poll the read model for the updated state for up to N seconds. (3) WebSocket/SSE - server pushes read model updates to the UI when the projection processes the event. (4) Accept stale reads with a "refreshing..." indicator.

Q44. What is a snapshot in event sourcing and when should you use it?

A snapshot captures the current state of an aggregate after processing N events, stored alongside the event log. On loading, the system reads the latest snapshot then replays only events after that snapshot - avoiding full event replay. Use when: aggregates have many events (>50-100 typically), loading latency is unacceptable, or during startup after deployment. Snapshots do not replace events - both are kept.

Q45. How do you test CQRS+Event Sourcing applications?

Unit test commands: given(events) -> when(command) -> then(expected_events) using Axon Test Fixtures. Unit test projections: given(events) -> expect(read model state). Integration test: full flow from command to projection update. Test compensating transactions with failure injection. Property-based testing: any sequence of commands should leave the system in a consistent state.

Q46. What is the difference between 2PC in databases and distributed 2PC?

Database-level 2PC (XA transactions): supported by JDBC drivers and JTA (Atomikos, Bitronix). Coordinates across multiple databases in a single JVM transaction. Works but has performance overhead and coordinator failure risk. Distributed 2PC: used in distributed databases (e.g., Google Spanner uses TrueTime + Paxos for distributed commit). Spanner achieves external consistency with linearizability across shards.

Q47. How does Debezium implement CDC for MySQL?

Debezium connects to MySQL as a replica, reads the binlog (binary log) stream, and publishes row-level change events (before/after) to Kafka topics. It handles: schema changes (DDL captured in schema history topic), initial snapshot (reads existing data before log tailing), connector state persistence (offsets in Kafka), and filtering (include/exclude specific tables or columns).

Q48. What is the Write-Ahead Log (WAL) and its role in replication?

The WAL (PostgreSQL) or binlog (MySQL) is an append-only log of all database changes, written before applying to data pages. For replication: the primary streams WAL records to replicas which replay them to maintain a copy of the primary's state. CDC tools (Debezium) read the WAL non-invasively. WAL enables: point-in-time recovery, streaming replication, and logical replication (selective table replication).

Q49. What is a global secondary index in DynamoDB and how does it relate to sharding?

DynamoDB shards data by partition key. Queries must include the partition key. For queries on other attributes, a Global Secondary Index (GSI) creates a separate partition of the data indexed by a different key. Internally, DynamoDB replicates the relevant data to GSI partitions asynchronously. Similar concept to maintaining a separate "index shard" for cross-shard queries on secondary keys.

Q50. How do you monitor sharding health in production?

Key metrics: per-shard query latency and throughput (watch for hotspot - one shard at 80% while others at 20%), shard size growth (detect uneven distribution), cross-shard query count and latency, replication lag per shard. Tools: Vitess dashboard, ProxySQL stats, Percona Monitoring and Management (PMM), custom Prometheus metrics from shard router. Alert on shard imbalance > 20%.

Q51. What is Citus and how does it enable PostgreSQL sharding?

Citus is a PostgreSQL extension that transforms Postgres into a distributed database. It supports: hash and range distribution of tables across worker nodes, parallel query execution, distributed DDL, and a coordinator node that routes queries. Co-location: related tables sharded by the same key are co-located on the same worker, enabling efficient local joins without cross-worker data movement.

Q52. What challenges arise when adding a new shard to an existing sharded cluster?

Challenges: (1) Data migration - must move a subset of existing data to the new shard without downtime; (2) Routing update - shard router must be updated atomically; (3) Double-write period - writes may need to go to both old and new shards during migration; (4) Read cutover - switch reads to new shard only after data sync is complete; (5) Rollback plan if migration fails. Online resharding tools (Vitess, Citus) automate this process.
